DATABASE NORMALIZATION

Complete Study Notes and Practical SQL Guide

Beginner → Intermediate → Professional

1NF • 2NF • 3NF • BCNF • 4NF • 5NF • DKNF

Functional Dependencies • Keys • Anomalies • SQL • Design • Exercises

Purpose: A university-level reference for learning, revising, designing and implementing normalized relational databases.

SQL: Examples use generally portable SQL; exact syntax can vary slightly among MySQL, PostgreSQL, SQL Server, Oracle and SQLite.

Table of Contents

1. Database fundamentals and relational design
2. Why normalization is necessary
3. Relational terminology
4. Keys and candidate keys
5. Functional dependencies
6. Data anomalies
7. First Normal Form (1NF)
8. Second Normal Form (2NF)
9. Third Normal Form (3NF)
10. Boyce-Codd Normal Form (BCNF)
11. Fourth Normal Form (4NF)
12. Fifth Normal Form (5NF/PJNF)
13. Domain-Key Normal Form (DKNF)
14. Normal forms comparison
15. Professional normalization workflow
16. University registration case study
17. Sales/order case study
18. Clinic case study
19. SQL implementation
20. Constraints and integrity
21. Normalization versus denormalization
22. Lossless decomposition and dependency preservation
23. Common mistakes
24. Advanced professional considerations
25. Exam/interview questions
26. Practical exercises

27. Revision checklist
28. Glossary
29. Final summary

1. Database Fundamentals and Relational Design

A database is an organized collection of related data. A relational database represents data as relations, normally implemented as tables. A relational DBMS provides facilities for defining structures, inserting data, querying data, enforcing integrity, controlling transactions and managing security.

Concept	Explanation
Database	Organized collection of related data.
DBMS	Software used to create, store, retrieve, update and manage databases.
RDBMS	DBMS based on the relational model.
Relation	A table in the relational model.
Tuple	A row/record in a relation.
Attribute	A column/property.
Schema	The logical structure and definitions of a database.
Domain	The valid set/type of values for an attribute.

Why design matters

- A database should represent real-world facts accurately.
- Each fact should have a clear owner.
- Relationships should be represented through keys.
- Business rules should be enforceable.

- The design should minimize contradictory copies of the same fact.

2. Why Normalization Is Necessary

Normalization is a formal method of structuring relational tables so that attributes and relationships are represented correctly and unnecessary redundancy is controlled.

Main objectives

- Reduce unnecessary redundancy.
- Prevent insert, update and delete anomalies.
- Improve consistency and integrity.
- Make dependencies explicit.
- Make constraints easier to enforce.
- Produce maintainable logical schemas.

Redundancy is not simply repeated text

A value can legitimately occur in many rows. For example, a CourseID can occur in many Enrollment rows because many students may take the course. The problem occurs when the same fact is copied repeatedly and must be kept synchronized.

3. Relational Terminology

Term	Professional meaning
Cardinality	Number of tuples/rows in a relation; in ER modeling it can also describe relationship multiplicity.
Degree	Number of attributes/columns

	in a relation.
Integrity constraint	Rule restricting invalid database states.
Entity	A distinguishable real-world object or concept.
Relationship	Association between entities.
Decomposition	Splitting a relation into smaller relations.
Join	Operation that combines related rows.
Prime attribute	Attribute belonging to at least one candidate key.
Non-prime attribute	Attribute belonging to no candidate key.

4. Keys and Candidate Keys

4.1 Superkey

A superkey is any set of attributes that uniquely identifies a row. If StudentID is unique, both {StudentID} and {StudentID, StudentName} are superkeys, but the latter contains unnecessary information.

4.2 Candidate key

A candidate key is a minimal superkey. No attribute can be removed without losing uniqueness.

4.3 Primary key

The primary key is the candidate key selected as the main identifier. It should normally be unique and NOT NULL.

4.4 Composite key

A composite key contains multiple attributes. Enrollment(StudentID, CourseID) is a common example.

4.5 Foreign key

A foreign key references a key in another table and enforces referential integrity.

Key type	Example	Meaning
Superkey	{StudentID, Name}	Unique, but may not be minimal.
Candidate key	{StudentID}	Minimal unique identifier.
Primary key	StudentID	Chosen candidate key.
Composite key	{StudentID, CourseID}	Multiple attributes identify one relationship.
Foreign key	Enrollment.StudentID	References Student.StudentID.

5. Functional Dependencies

Functional dependency is the central reasoning tool of normalization. X → Y means that whenever two rows have the

same X value, they must have the same Y value. X is the determinant and Y is functionally dependent on X.

```
StudentID → StudentName, ProgrammeID
ProgrammeID → ProgrammeName
CourseID → CourseName, DepartmentID
DepartmentID → DepartmentName
(StudentID, CourseID) → Grade
```

5.1 Trivial and non-trivial dependencies

- Trivial: X → Y where Y is a subset of X. Example: {A,B} → A.
- Non-trivial: Y is not a subset of X. Example: StudentID → StudentName.

5.2 Full functional dependency

Y is fully dependent on X when Y depends on the whole determinant and not on any proper subset of it.

5.3 Partial dependency

A non-key attribute has a partial dependency when it depends on only part of a composite candidate key. Partial dependencies are the central issue of 2NF.

5.4 Transitive dependency

A common pattern is X → Y and Y → Z, causing X → Z indirectly. If Y is not a candidate key in the relation, this can violate 3NF.

5.5 Armstrong's axioms

- Reflexivity: if Y ⊆ X, then X → Y.
- Augmentation: if X → Y, then XZ → YZ.
- Transitivity: if X → Y and Y → Z, then X → Z.

- Union: X → Y and X → Z imply X → YZ.
- Decomposition: X → YZ implies X → Y and X → Z.
- Pseudotransitivity: X → Y and WY → Z imply WX → Z.

5.6 Attribute closure

The closure X+ is the set of all attributes functionally determined by X. It is used to test superkeys and find candidate keys.

```
F = { A → B, B → C, AC → D }

A+:
Start {A}
A → B          => {A,B}
B → C          => {A,B,C}
AC → D         => {A,B,C,D}

Therefore A+ = {A,B,C,D}.
If these are all attributes of R, A is a
superkey.
```

6. Data Anomalies

StudentID	StudentName	CourseID	CourseName	LecturerID	LecturerName
S001	Mwamba	C101	Database	L01	Dr Banda
S001	Mwamba	C102	Data Mining	L02	Ms Phiri
S002	Chanda	C101	Database	L01	Dr Banda

6.1 Update anomaly

If Dr Banda changes phone number, every row containing L01 must be updated. Missing one creates inconsistent data.

6.2 Insert anomaly

A new course may be impossible to insert without inventing a student if course facts are stored only inside registration rows.

6.3 Delete anomaly

Deleting the final student's enrollment in a course can accidentally delete the only stored description of that course.

Key idea: Normalization separates facts so that changing, adding or deleting one fact does not unintentionally alter another fact.

7. First Normal Form (1NF)

A relation is in 1NF when its attributes contain values that are atomic with respect to the chosen relational design and repeating groups are eliminated.

Unnormalized example

StudentID	StudentName	Courses
S001	Mwamba	Database, Data Mining, Statistics
S002	Chanda	Database

1NF version

StudentID	StudentName	Course
S001	Mwamba	Database

S001	Mwamba	Data Mining
S001	Mwamba	Statistics
S002	Chanda	Database

Typical SQL

```
CREATE TABLE StudentCourse_1NF (
    StudentID   VARCHAR(20) NOT NULL,
    StudentName VARCHAR(100) NOT NULL,
    CourseName  VARCHAR(100) NOT NULL,
    PRIMARY KEY (StudentID, CourseName)
);
```

- Avoid comma-separated lists used as multiple values.
- Avoid Phone1, Phone2, Phone3 when the number of phones is not fixed.
- Each row should represent one occurrence of the relation.
- Attributes should have consistent domains and meanings.

8. Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on every candidate key. In beginner cases, think: remove partial dependencies on part of a composite key.

StudentID	CourseID	StudentName	CourseName	Grade
S001	C101	Mwamba	Database	A
S001	C102	Mwamba	Data Mining	B
S002	C101	Chanda	Database	A

Assume (StudentID, CourseID) is the key. StudentName depends only on StudentID, while CourseName depends only on CourseID. Therefore both are partial dependencies.

Decomposition

Table	Attributes
Student	StudentID PK, StudentName
Course	CourseID PK, CourseName
Enrollment	StudentID PK/FK, CourseID PK/FK, Grade

```
CREATE TABLE Student (
    StudentID VARCHAR(20) PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL
);

CREATE TABLE Course (
    CourseID VARCHAR(20) PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL
);

CREATE TABLE Enrollment (
    StudentID VARCHAR(20) NOT NULL,
    CourseID VARCHAR(20) NOT NULL,
    Grade VARCHAR(5),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
```

```
    FOREIGN KEY (CourseID) REFERENCES
Course(CourseID)
);
```

Important: If the candidate key has only one attribute, there cannot be a partial dependency on part of that key.

9. Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and, for every non-trivial functional dependency X → A, either X is a superkey or A is a prime attribute.

Beginner interpretation

A useful memory phrase is: 'the attributes depend on the key, the whole key, and nothing but the key.' The phrase is a teaching aid, while the formal definition is more precise.

```
StudentID → ProgrammeID
ProgrammeID → ProgrammeName

Therefore StudentID → ProgrammeName indirectly.
ProgrammeName belongs in Programme, not Student.
```

Student	Programme
StudentID PK, StudentName, ProgrammeID FK	ProgrammeID PK, ProgrammeName UNIQUE

```
CREATE TABLE Programme (
    ProgrammeID INT PRIMARY KEY,
    ProgrammeName VARCHAR(150) NOT NULL UNIQUE
```

```
);

CREATE TABLE Student (
    StudentID VARCHAR(20) PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    ProgrammeID INT NOT NULL,
    FOREIGN KEY (ProgrammeID) REFERENCES
Programme(ProgrammeID)
);
```

Exam distinction: 2NF removes partial dependencies. 3NF removes inappropriate transitive dependencies. BCNF is stricter than 3NF.

10. Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency X → Y, X is a superkey.

BCNF handles some redundancy that can remain in 3NF, especially where there are multiple candidate keys.

Typical example

Teaching(Student, Course, Instructor), with rules such as Instructor → Course and (Student, Course) → Instructor. If Instructor is not a superkey, Instructor → Course violates BCNF.

BCNF decomposition strategy

30. Find a non-trivial dependency X → Y whose determinant X is not a superkey.
31. Create R1 containing X and Y.

32. Create R2 containing the remaining attributes plus X.
33. Check the new relations for lossless join and BCNF.
34. Repeat if necessary.

Core relationship: Every BCNF relation is in 3NF, but a 3NF relation is not necessarily in BCNF.

11. Fourth Normal Form (4NF)

4NF addresses multivalued dependencies. A relation is in 4NF if, for every non-trivial multivalued dependency $X \twoheadrightarrow Y$, X is a superkey.

Suppose a lecturer has several research areas and several office locations, and the two sets are independent.

Lecturer	ResearchArea	Office
L01	AI	Block A
L01	AI	Block B
L01	Data Mining	Block A
L01	Data Mining	Block B

The rows represent combinations of two independent sets. The better design is:

Table	Columns
LecturerResearch	Lecturer, ResearchArea
LecturerOffice	Lecturer, Office

Memory aid: 4NF = do not combine independent multi-valued facts into one relation merely because they share the same determinant.

12. Fifth Normal Form (5NF / Project-Join Normal Form)

5NF addresses join dependencies. A relation is in 5NF when every non-trivial join dependency is implied by candidate keys.

- 5NF is an advanced relational design concept.
- It becomes relevant with complex many-to-many-to-many business rules.
- The purpose is to avoid redundancy that can only be understood through joins.
- Incorrect decomposition can create spurious combinations.
- Most ordinary application schemas are designed successfully at 3NF or BCNF, with 5NF considered when the business rules justify it.

13. Domain-Key Normal Form (DKNF)

DKNF is a theoretical ideal in which every constraint on a relation is a logical consequence of domain constraints and key constraints.

Real systems contain business rules that may not be expressible solely through domains and keys, so DKNF is mainly useful as an advanced theoretical concept.

Practical approach: Strong keys, foreign keys, UNIQUE, CHECK, NOT NULL, transactions and application/business rules provide the practical integrity framework.

14. Normal Forms Comparison

Form	Requirement	Main problem addressed
UNF	May contain repeating/non-atomic groups	Poor structure
1NF	Atomic values and no repeating groups	Repeating/multi-valued representation
2NF	1NF + no partial dependency of non-prime attributes	Partial dependency
3NF	2NF + formal FD condition	Transitive/non-key dependency
BCNF	Every determinant is a superkey	Stronger FD redundancy control
4NF	Every non-trivial MVD has a superkey determinant	Independent multivalued facts
5NF	Every non-trivial JD is implied by candidate keys	Complex join dependencies
DKNF	Constraints derive from domains and keys	Theoretical ideal

Memory aid: 1NF = atomic; 2NF = whole key; 3NF = nothing but the key; BCNF = every determinant is a key; 4NF = independent multivalued facts; 5NF = join dependencies.

15. Professional Normalization Workflow

35. Understand the business scope and requirements.
36. Identify entities, relationships and business rules.
37. List attributes and define their meanings.
38. Identify all candidate keys.
39. Document functional dependencies.
40. Convert repeating groups to 1NF.
41. Remove partial dependencies to achieve 2NF.
42. Remove inappropriate transitive dependencies to achieve 3NF.
43. Test for BCNF violations where multiple candidate keys exist.
44. Investigate multivalued dependencies for 4NF.
45. Investigate join dependencies for 5NF when necessary.
46. Verify lossless decomposition.
47. Evaluate dependency preservation.
48. Implement PK, FK, UNIQUE, NOT NULL and CHECK constraints.
49. Test realistic insert, update, delete and query operations.
50. Index based on measured workload.
51. Only then consider deliberate denormalization if justified.

Questions for every attribute

- What real-world fact does this represent?
- Which entity owns the fact?
- What determines its value?
- Does it depend on the whole candidate key?
- Is it indirectly determined by another non-key attribute?
- Can it have multiple independent values?
- Could deleting one relationship accidentally delete another fact?

16. Worked Case Study: University Registration

Business rules: a student belongs to one programme; a programme has a unique name; a student can enroll in many courses; a course can have many students; an enrollment has a grade; a course belongs to one department.

Initial relation

```
Registration(
 StudentID, StudentName, ProgrammeID,
ProgrammeName,
 CourseID, CourseName, DepartmentID,
DepartmentName, Grade
)
```

Functional dependencies

```
StudentID → StudentName, ProgrammeID
ProgrammeID → ProgrammeName
CourseID → CourseName, DepartmentID
```

```
DepartmentID → DepartmentName
(StudentID, CourseID) → Grade
```

Normalized tables

Table	Primary key	Foreign keys
Programme	ProgrammeID	—
Student	StudentID	ProgrammeID
Department	DepartmentID	—
Course	CourseID	DepartmentID
Enrollment	StudentID + CourseID	StudentID, CourseID

Complete SQL

```
CREATE TABLE Programme (
    ProgrammeID INT PRIMARY KEY,
    ProgrammeName VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE Student (
    StudentID VARCHAR(20) PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    ProgrammeID INT NOT NULL,
    FOREIGN KEY (ProgrammeID) REFERENCES
Programme(ProgrammeID)
);

CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(150) NOT NULL UNIQUE
```

```
);

CREATE TABLE Course (
    CourseID VARCHAR(20) PRIMARY KEY,
    CourseName VARCHAR(150) NOT NULL,
    DepartmentID INT NOT NULL,
    FOREIGN KEY (DepartmentID) REFERENCES
Department(DepartmentID)
);

CREATE TABLE Enrollment (
    StudentID VARCHAR(20) NOT NULL,
    CourseID VARCHAR(20) NOT NULL,
    Grade VARCHAR(5),
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES
Course(CourseID)
);
```

Query across normalized tables

```
SELECT s.StudentID, s.StudentName,
p.ProgrammeName,
       c.CourseName, e.Grade
FROM Enrollment AS e
JOIN Student AS s ON e.StudentID = s.StudentID
JOIN Programme AS p ON s.ProgrammeID =
p.ProgrammeID
```

```
JOIN Course AS c ON e.CourseID = c.CourseID
ORDER BY s.StudentID, c.CourseName;
```

17. Worked Case Study: Sales and Orders

An online shop has customers, orders and products. Orders and products have a many-to-many relationship, so OrderItem resolves it.

Table	Columns
Customer	CustomerID, CustomerName, Email
OrderHeader	OrderID, CustomerID, OrderDate, Status
Product	ProductID, ProductName, CurrentUnitPrice
OrderItem	OrderID, ProductID, Quantity, UnitPriceAtSale

Important: UnitPriceAtSale is a historical fact about the order line. It should not automatically change when Product.CurrentUnitPrice changes.

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(120) NOT NULL,
    Email VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE Product (
```

```
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(150) NOT NULL,
    CurrentUnitPrice DECIMAL(12,2) NOT NULL
CHECK (CurrentUnitPrice >= 0)
);

CREATE TABLE OrderHeader (
    OrderID INT PRIMARY KEY,
    CustomerID INT NOT NULL,
    OrderDate DATE NOT NULL,
    Status VARCHAR(30) NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID)
);

CREATE TABLE OrderItem (
    OrderID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    UnitPriceAtSale DECIMAL(12,2) NOT NULL CHECK
(UnitPriceAtSale >= 0),
    PRIMARY KEY (OrderID, ProductID),
    FOREIGN KEY (OrderID) REFERENCES
OrderHeader(OrderID),
    FOREIGN KEY (ProductID) REFERENCES
Product(ProductID)
);
```

```
SELECT o.OrderID,
       SUM(oi.Quantity * oi.UnitPriceAtSale) AS
OrderTotal
FROM OrderHeader AS o
JOIN OrderItem AS oi ON o.OrderID = oi.OrderID
GROUP BY o.OrderID;
```

18. Worked Case Study: Clinic/Hospital

Entity	Typical attributes
Patient	PatientID, Name, DateOfBirth, Phone
Doctor	DoctorID, Name, SpecialtyID
Specialty	SpecialtyID, SpecialtyName
Appointment	AppointmentID, PatientID, DoctorID, DateTime, Reason
Medication	MedicationID, MedicationName
Prescription	PrescriptionID, AppointmentID, MedicationID, Dose, Frequency, Duration

Prescription is a relationship entity: one appointment can involve many medications, and a medication can be prescribed in many appointments.

19. SQL Implementation of Normalized Designs

19.1 INSERT

```
INSERT INTO Programme (ProgrammeID,
ProgrammeName)
VALUES (1, 'BSc Data Science');

INSERT INTO Student (StudentID, StudentName,
ProgrammeID)
VALUES ('S001', 'Mwamba Perseverance', 1);
```

19.2 UPDATE

```
UPDATE Student
SET StudentName = 'Mwamba P. Mwamba'
WHERE StudentID = 'S001';
```

19.3 DELETE

```
DELETE FROM Enrollment
WHERE StudentID = 'S001'
  AND CourseID = 'C101';
```

19.4 JOIN

```
SELECT s.StudentName, c.CourseName, e.Grade
FROM Enrollment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Course c ON e.CourseID = c.CourseID;
```

19.5 GROUP BY

```
SELECT CourseID, COUNT(*) AS NumberOfStudents
FROM Enrollment
GROUP BY CourseID;
```

20. Constraints and Integrity

Constraint	Purpose	Example
PRIMARY KEY	Unique row identifier	PRIMARY KEY (StudentID)
FOREIGN KEY	Referential integrity	REFERENCES Programme(ProgrammeID)
UNIQUE	No duplicate value/combinations	UNIQUE (Email)
NOT NULL	Value required	Name VARCHAR(100) NOT NULL
CHECK	Business condition	CHECK (Quantity > 0)
DEFAULT	Default value	Status VARCHAR(20) DEFAULT 'ACTIVE'

```
CREATE TABLE Enrollment (
    StudentID VARCHAR(20) NOT NULL,
    CourseID VARCHAR(20) NOT NULL,
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID)
        REFERENCES Student(StudentID)
        ON DELETE RESTRICT,
    FOREIGN KEY (CourseID)
        REFERENCES Course(CourseID)
        ON DELETE RESTRICT
);
```

Use referential actions deliberately. CASCADE can be useful, but it can also remove many dependent rows unexpectedly if the business rule does not permit that behavior.

21. Normalization versus Denormalization

Normalization	Denormalization
Reduces unnecessary redundancy.	Intentionally introduces selected redundancy.
Strong fit for transactional integrity.	Can help selected analytical/read-heavy workloads.
Often produces more tables and joins.	Can reduce joins or precompute values.
Starts from logical correctness.	Should be justified by measured requirements.

When denormalization may be justified

- Measured performance problems.
- Reporting/analytics patterns where repeated joins are expensive.
- Materialized views or summary tables.
- Controlled duplication with synchronization rules.
- Data warehouse dimensional modeling.

Professional rule: Normalize first, optimize second. Never denormalize merely because a schema has many tables.

22. Lossless Decomposition and Dependency Preservation

22.1 Lossless decomposition

A decomposition is lossless when joining the resulting relations reconstructs the intended original information without losing valid facts or creating spurious tuples.

22.2 Dependency preservation

A decomposition is dependency-preserving when the important functional dependencies can be enforced using the decomposed relations without requiring joins.

22.3 Why both matter

- Losslessness protects information correctness.
- Dependency preservation makes integrity rules easier to enforce.
- 3NF synthesis commonly provides useful lossless and dependency-preserving designs.
- BCNF may sometimes require a trade-off involving dependency preservation.

23. Common Database Normalization Mistakes

Mistake	Problem	Better practice
One giant table	Mixes unrelated facts and duplicates data.	Separate entities and relationships.
Comma-separated values	Hard to validate/query; violates intended atomic representation.	Use child/relationship rows.
Phone1/Phone2/Phone3	Artificial limit and awkward queries.	Use a related Phone table if needed.
No foreign keys	Invalid references can	Enforce referential integrity.

	enter the database.	
Names as identifiers	Names may change or not be unique.	Use suitable stable keys; enforce business-key UNIQUE rules.
Denormalizing too early	Creates duplication before correctness.	Normalize first.
Ignoring business rules	Wrong dependencies lead to wrong tables.	Document rules before decomposition.
Assuming repeated values are always bad	Legitimate relationships can repeat identifiers.	Distinguish repeated relationships from duplicated facts.

24. Advanced Professional Considerations

24.1 Natural keys and surrogate keys

Natural keys have business meaning, such as an ISBN. Surrogate keys are artificial identifiers such as integers or UUIDs. A surrogate key does not remove the need for UNIQUE constraints on genuine business identifiers.

```
CREATE TABLE Customer (
    CustomerID BIGINT PRIMARY KEY,
    Email VARCHAR(255) NOT NULL UNIQUE,
```

```
    CustomerName VARCHAR(150) NOT NULL
);
```

24.2 NULL

NULL is not zero and is not necessarily an empty string. It represents missing, unknown or inapplicable information depending on context. Excessive NULLs can indicate that unrelated facts have been combined.

24.3 Derived attributes

OrderTotal can be calculated from OrderItem rows. Storing it can be valid for performance or auditing, but then consistency must be controlled.

24.4 Historical/temporal data

Current Product price and historical OrderItem price are different facts because they refer to different times.

24.5 Indexes

Normalization is about logical structure; indexing is about access performance. Indexes should be selected based on actual query patterns, cardinality and measured workload.

24.6 Transactions

```
BEGIN;

INSERT INTO OrderHeader (OrderID, CustomerID,
OrderDate, Status)
VALUES (1001, 7, CURRENT_DATE, 'NEW');
```

```
INSERT INTO OrderItem (OrderID, ProductID,
Quantity, UnitPriceAtSale)
VALUES (1001, 20, 2, 49.99);

COMMIT;
```

A transaction should leave the database in a valid state. If an operation fails, the DBMS can roll it back rather than leaving only part of the intended change.

24.7 OLTP versus data warehousing

OLTP systems often favor normalized designs for frequent inserts, updates and strong consistency. Data warehouses often use dimensional models such as star schemas, where controlled redundancy is intentional for analytical performance. The different designs serve different workloads.

25. Examination and Interview Questions

25.1 Short-answer questions

52. What is normalization and why is it performed?
53. Define relation, tuple, attribute and domain.
54. Differentiate superkey, candidate key and primary key.
55. What is a composite key?
56. What is a foreign key?
57. Define functional dependency.
58. What is a determinant?
59. Differentiate full, partial and transitive dependency.
60. State the requirements of 1NF, 2NF and 3NF.

61. State the formal BCNF condition.
62. What problem does 4NF solve?
63. What is a multivalued dependency?
64. What is 5NF?
65. What is a join dependency?
66. What is DKNF?
67. Explain insertion, update and deletion anomalies.
68. What is lossless decomposition?
69. What is dependency preservation?
70. Why can a single-attribute key not have a partial dependency?
71. Why is BCNF stronger than 3NF?

25.2 Long-answer questions

72. Normalize a university registration relation from an unnormalized form to 3NF and provide SQL.
73. Explain functional dependencies and calculate an attribute closure step by step.
74. Compare 1NF, 2NF, 3NF and BCNF with examples.
75. Explain insertion, update and deletion anomalies and how decomposition prevents them.
76. Explain 4NF using independent multivalued facts.
77. Explain lossless decomposition and dependency preservation.
78. Compare normalization and denormalization in OLTP and data warehousing.
79. Design a normalized online shop database and implement it in SQL.

80. Explain why splitting a table does not automatically prove that it is normalized.
81. Discuss natural keys, surrogate keys, constraints and indexing in professional database design.

26. Practical SQL and Design Exercises

Exercise 1 — Student registration

Given StudentID, StudentName, ProgrammeID, ProgrammeName, CourseID, CourseName, LecturerID, LecturerName and Grade, identify candidate keys and dependencies, normalize to 3NF, and write CREATE TABLE statements.

Exercise 2 — Online store

Create Customer, Product, OrderHeader and OrderItem tables with PK, FK, UNIQUE, NOT NULL and CHECK constraints.

Exercise 3 — Clinic

Create Patient, Doctor, Appointment, Medication and Prescription tables. Explain the relationship between Appointment and Medication.

Exercise 4 — Library

Create Member, Book, Author, BookAuthor and Loan. Explain why BookAuthor is needed when books can have multiple authors.

Exercise 5 — Dependency analysis

R(EmployeeID, EmployeeName, DepartmentID, DepartmentName,
ManagerID, ManagerName, ProjectID, HoursWorked)

Assume:
EmployeeID → EmployeeName, DepartmentID
DepartmentID → DepartmentName, ManagerID
ManagerID → ManagerName
(EmployeeID, ProjectID) → HoursWorked

Tasks:
1. Identify the candidate key.
2. Identify partial dependencies.
3. Identify transitive dependencies.
4. Decompose to 3NF.
5. Write SQL.

Exercise 6 — Query your normalized database

Write SQL to:
1. List all students and programmes.
2. List all courses and departments.
3. List each student's courses and grades.
4. Count students per programme.
5. Count enrollments per course.
6. Find students with no enrollment.
7. Find courses with no enrollment.
8. Update a student's programme.

```
9. Delete one enrollment.
10. Attempt an invalid foreign-key insert and
explain the result.
```

Exercise 7 — Advanced closure

```
Given R(A,B,C,D,E) and:
F = {A → BC, B → D, CD → E}

Find A+.
Determine whether A is a superkey.
Determine whether A is a candidate key.
Show each inference step.
```

27. Comprehensive Revision Checklist

- I can define database, DBMS, RDBMS, relation, tuple, attribute and domain.
- I can distinguish superkey, candidate key, primary key, composite key and foreign key.
- I can write and interpret X → Y functional dependencies.
- I understand trivial, non-trivial, full, partial and transitive dependencies.
- I can calculate attribute closure.
- I can identify insertion, update and deletion anomalies.
- I can convert an unnormalized relation to 1NF.
- I can remove partial dependencies for 2NF.
- I can remove transitive dependencies for 3NF.
- I understand the formal 3NF rule.
- I can explain why BCNF is stronger than 3NF.

- I understand multivalued dependencies and 4NF.
- I understand join dependencies and 5NF.
- I understand DKNF as an advanced theoretical concept.
- I can reason about lossless decomposition.
- I understand dependency preservation.
- I can implement PK, FK, UNIQUE, NOT NULL and CHECK constraints.
- I understand normalization versus denormalization.
- I can design a normalized schema from real business requirements.

28. Glossary

Term	Definition
Anomaly	Problem caused by unsuitable relational structure.
Atomic	Represented as one value according to the design's intended domain.
BCNF	Normal form requiring every determinant of a non-trivial FD to be a superkey.
Candidate key	Minimal attribute set that uniquely identifies a row.
Composite key	Key made of two or more attributes.
Decomposition	Splitting a relation into smaller relations.
Determinant	Left side of a functional dependency.

Domain	Set of valid values for an attribute.
Foreign key	Attribute set referencing a key in another relation.
Functional dependency	Constraint represented as X → Y.
Join dependency	Constraint about reconstructing a relation from projections/joins.
Lossless	Decomposition that preserves the intended information when relations are joined.
Multivalued dependency	X ↠ Y expresses independent multiple Y values associated with X.
Normalization	Formal organization of relational tables using normal forms.
Partial dependency	Non-prime attribute depends on part of a composite candidate key.
Prime attribute	Attribute appearing in at least one candidate key.
Referential integrity	Foreign-key references must satisfy the parent-key rule.
Superkey	Any attribute set that uniquely identifies a row.
Transitive dependency	Indirect dependency through another attribute.

29. Final Summary

Normalization is not simply 'making tables smaller'. It is a disciplined method of deciding where facts belong and how relationships should be represented. The process begins with requirements and business rules, identifies keys and dependencies, and then applies increasingly strict normal forms.

1NF establishes atomic relational representation. 2NF removes partial dependencies. 3NF removes inappropriate transitive dependencies. BCNF requires every determinant in a non-trivial functional dependency to be a superkey. 4NF handles independent multivalued facts. 5NF handles advanced join dependencies. DKNF is a theoretical ideal based on domain and key constraints.

A professional designer should normalize for correctness first, enforce integrity through constraints, optimize with indexes and query design, use transactions for atomic operations, and only introduce deliberate redundancy when a measurable architectural or performance requirement justifies it.

Core memory rule: A fact should be stored in the relation whose key naturally determines that fact, rather than repeatedly copied because of an unrelated relationship.

Study strategy: For examinations, learn both the definitions and the reasoning: identify keys → write dependencies → find

anomalies → decompose → verify the normal form → implement with SQL.

Selected References

- Elmasri, R. & Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F. & Sudarshan, S. Database System Concepts. McGraw-Hill.
- Date, C. J. An Introduction to Database Systems. Addison-Wesley.
- Connolly, T. & Begg, C. Database Systems: A Practical Approach to Design, Implementation, and Management. Pearson.
- Coronel, C. & Morris, S. Database Systems: Design, Implementation, & Management. Cengage.